ASSESSMENT 2 - Project
AWS Cloud System Development

	Assessment Type: Group assignment (**maximum of four students**), **individual work is NOT allowed.** Submit your solutions online via Canvas>Assignments>**Assessment 2.** Clarifications/updates will be made via announcements and **relevant discussion forums**.
	Due date: **11:59PM on Friday, 8th May 2026.** A university standard late penalty of **10% (= 10 marks)** per day applies for the days that you are late to submit your work for, unless a special consideration or an extension has been granted.
	Weighting: This is a group assignment (100 marks) that contributes 40% of the total assessment.

Overview

Develop a cloud-based web application leveraging AWS S3, API Gateway, Lambda, DynamoDB, and compute services such as EC2 or ECS.

This assessment will evaluate your ability to architect, develop, and deploy cloud-based applications using AWS EC2 or ECS, S3, API Gateway, Lambda, and DynamoDB services.

Learning Outcomes

By completing this assessment, students will be able to **design, develop, and deploy a cloud-based web application** by effectively integrating key AWS services.

Group Assessment

Groups must be formed by up to 4 students (please use the online forum to connect with classmates**).**

All members are expected to contribute equally. Rather than dividing the work and working in isolation, you're expected to **collaborate on all tasks** and **cross-check each other's work at every step**. **A group work log is mandatory and must be submitted as part of your final submission.** The work log should clearly document each member's contributions and participation. **The work log template is provided on the assignment page and must be used.** Unless a contribution dispute is formally reported and supported by evidence in the submitted work log, all group members will receive the same grade.

Note: Individual work is **NOT permitted**. Students who do not form a group by the end of Week 6 (19/04) will be randomly assigned to a group of four.

Group Formation and Locking Policy

- Navigate to **People** in Canvas, then select the Groups tab.
- Students must join an empty **A2 group** with their group members.
- You are NOT permitted to create new groups. You are NOT permitted to change the pre-defined group number. This restriction is in place because student-created groups are not linked to the grading system, which means they cannot be correctly identified by the system during marking.
- **Groups will be locked on 7 May at 11:59 PM, which is 24 hours before the assignment deadline.** Once groups are locked, students can no longer join, leave, or modify their groups. This ensures consistency in grading and prevents last-minute changes that may affect group accountability.

You are required to form your group before beginning the assessment. Students who do not correctly form their group before submission will receive a 10% deduction (equivalent to 10 marks for A2).

Group Leadership

The first student to join a group on Canvas is assigned as the group leader by default. However, this role can be changed later within the group settings if needed.

The group leader is responsible for ensuring that the group is correctly finalized before the deadline. This includes:

- Confirming all required files are prepared and correctly named.
- Verifying the correct members are listed in the group.
- Submitting the final work before the deadline.
- Keeping and submitting the group work log as part of the submission.

Extension Policy for Group Assignments

Any extension requests for group assignments must be approved by the Course Coordinator.

Group extensions will only be considered under exceptional circumstances (e.g., fewer than **two** group members remain able to contribute at the same time) and strictly require supporting documentation.

If you intend to apply for an individual extension, you must withdraw from the group and send a written email to all group members (cc the teaching team) confirming your withdrawal. If your individual application is approved, any alternative tasks will be provided only to you, not to your former group members.

What to Do If a Team Member Isn't Contributing

- Try to resolve the issue first through communication within the group.
- Document communication attempts (e.g., Canvas group space, emails, messages). Keep a simple log of who completed which parts of the work.
- If the situation doesn't improve, you may choose to leave the group, join other groups OR complete the assignment individually.

Please notify the tutor and the course coordinator early if you decide to do so. Adjustments due to non-contributing members will not be granted unless clear evidence is provided.

Assessment Details

Create a simple online music subscription web application using AWS services.

Your assessment will be marked during your demo time (further details about the demo will be provided in Week 9). Marks will be assigned based on two key criteria:

1. The functional correctness of your application; and
2. The quality, coherence, and justification of your architectural design, including the design and rationale of your database key schema.

Database (DynamoDB) and Storage (S3) Initialisation

1. Write a program to create a **login** table in DynamoDB containing 10 entities with the following attributes and values.

email (Type: String)	user_name (Type: String)	password (Type: String)
s3######0@student.rmit.edu.au i.e., your RMIT student id+'0@student.rmit.edu.au'	FirstnameLastname0 i.e., your name+'0'	012345
s3######1@student.rmit.edu.au i.e., your RMIT student id+'1@student.rmit.edu.au'	FirstnameLastname1 i.e., your name+'1'	123456
...	...	...
s3######9@student.rmit.edu.au i.e., your RMIT student id+'9@student.rmit.edu.au'	FirstnameLastname9 i.e., your name+'9'	901234

For this specific cloud infrastructure assignment, storing passwords in plain text (as supplied in the dataset) is permitted for simplicity. Nevertheless, students must recognise that plain-text password storage is unacceptable in real-world systems. In production environments, passwords must always be securely **salted and hashed**.

2. Write a program to create a table titled **music** in DynamoDB with the following attributes: **title**, **artist**, **year**, **album**, and **image_url**.

---

3. Write a program to load the data from **2026a1.json** to your **music** table.

Before creating your table, you must analyse the raw data within the 2026a1.json dataset to understand the specific cardinality and relationships between song titles, artists, and albums. You must meticulously design your DynamoDB key schema based on these insights. The data populated into your **music** table must be a perfectly aligned, lossless representation of the raw JSON data, ensuring no songs are accidentally overwritten during the import.

---

4. Write a program that automatically downloads all artist images based on the image_url values found in **2026a1.json** and then uploads these images to an S3 bucket.

Web Application Functional Requirements

1. Login Page

The login page contains an **Email** text field, a **Password** field, and a **Login** button as well as a **register** link.

- When user clicks the **Login** button, it will validate if the user-entered credentials match with the information stored in the **login** table.
- If the user credential is invalid, the login page will display "**email or password is invalid**"; it will be redirected to the **main** page otherwise.

---

2 .Register Page

The **register** page contains an **Email** text field, a **Username** field, a **Password** field, and a **Register** button.

- When a user clicks the **Register** button, it will validate if the user-entered email matches with the email stored in the **login** table. (Note: Each registration must have a unique email address. The username is not intended to serve as a unique credential for authentication).
- If the entered email matches with the email stored in the **login** table, the register page will show "**The email already exists**".
- If the entered email is unique, the new user information will be stored in the **login** table, and the user will be redirected to the **login** page.

---

3. Main Page

The main page contains three areas (a **user area**, a **subscription area**, and a **query area**) and a **Logout** link. Displaying all songs stored in the database is not advisable, since in a real-world scenario it could contain millions of songs. For newly registered users, the subscription area must initially be empty and displayed in a clean state.

- After a user log in, the **user area** will show the corresponding **user_name**.
- **The subscription area w**ill show all the user subscribed music information stored in DynamoDB.
    - Each music information is followed by **the corresponding artist image retrieved from S3** and a "**Remove**" button.
    - If the user clicks "**Remove**", the corresponding information will be removed from the subscription area and the DynamoDB table.
- **The query area** should contain text areas for "**Title**", "**Year**", "**Artist**", "**Album**", and a "**Query**" button. At least one field must be completed.
    - If not contained in the table, show "**No result is retrieved. Please query again**".
    - If contained, show all retrieved music information. (Multiple conditions are connected by "AND" by default). Each is followed by the artist image from S3 and a "**Subscribe**" button. Clicking "**Subscribe**" adds it to the subscription area and stores it in DynamoDB.

      Note: During the demonstration, markers will provide specific search criteria, e.g., "Please find all songs by Taylor Swift in the album Fearless", “Please find all songs of Jimmy Buffett in 1974".
- When the user clicks **"Logout"**, they must be redirected to the login page. The user session should be terminated upon logout.

Note: While no marks will be deducted for basic aesthetics or an unpolished visual design, your web application must be highly functional, logically laid out, and user-friendly. It must successfully incorporate all required UI elements and error messages specified in the tasks.

Web Application Frontend Design

You are responsible for designing and implementing the frontend of the application.
If the frontend primarily consists of static assets (e.g., HTML, CSS, and JavaScript), you should carefully evaluate your hosting approach, considering factors such as scalability, operational simplicity, and cost-efficiency, rather than defaulting to a general-purpose compute service.

	In your report, you must **justify your chosen frontend hosting architecture**. Your reasoning should clearly explain **why your selected approach is appropriate for this workload**, and should discuss the key trade-offs (e.g., performance/latency, scalability under load, operational overhead, cost, security, and maintainability). Where relevant, briefly contrast your chosen approach with at least one alternative and explain why you did not select it.

Web Application Backend Design

The backend of the web application must be implemented **separately using all three architectural approaches** described below. Each backend implementation must be fully functional and capable of performing CRUD operations on both the DynamoDB **music** table and the **login** table.

1. **Amazon EC2** – A virtual server–based backend service.
2. **Amazon ECS** – A containerised backend deployment.
3. **API Gateway + AWS Lambda** – A fully serverless backend implementation.

Note: Each architecture must be deployed and validated **independently** to demonstrate functional equivalence.

	You are required to **critically compare and contrast** the three architectural approaches in your report. Your discussion should evaluate key factors such as scalability, cost efficiency, operational complexity, performance characteristics, maintainability, and suitability for different workload patterns. Based on your analysis, you must clearly identify and justify which architectural solution you consider most appropriate for this application scenario. Your conclusion should be supported by technical reasoning rather than personal preference.

Final Important Development Notes

1. When you use or adapt code developed by someone else, you must include an inline comment in the code citing your source.
2. The whole application must be **fully deployed in AWS**. Submissions that are not entirely deployed within AWS will receive **no marks**. This requirement ensures that students demonstrate practical competency in deploying, configuring, and integrating cloud-native services within a real cloud environment. The purpose of this assignment is not only to develop application functionality, but also to validate your ability to design and operate cloud infrastructure using AWS services. As this is a cloud infrastructure project, full AWS deployment is essential to meet the learning objectives and assessment criteria.
3. The application must be rendered correctly in a web browser when a user navigates directly to the **root URL** of the **public IPv4 DNS name** assigned to the EC2 instance or ECS task. Please make sure that

markers can access your web application via the standard HTTP(S) port. Using non-standard HTTP ports (i.e. ones other than 80/443) are NOT allowed.

4. **Using Elastic Beanstalk is NOT allowed as a valid means of deploying your application.**
5. AWS supports multiple programming languages; you can use any language(s) that you are familiar with. Your application still needs to fulfil the same requirements. You can use any framework providing you are writing the code for the core functions.
6. AWS Academy account does not allow to create any IAM roles. However, a role named LabRole has been pre-created for you. Use LabRole instead.
7. Students would also need to securely access the objects stored in S3. This is considered security best practice and should always be done.
8. Students must carefully design the DynamoDB key schema and indexing strategy, **including at least one Global Secondary Index (GSI) and one Local Secondary Index (LSI)**, to support efficient query patterns. Indexes should be purposefully designed rather than added arbitrarily.
9. **Both the Query and Scan operations** should be implemented appropriately for data retrieval from DynamoDB tables.
10. Please ensure that your implementation correctly handles different HTTP request methods (including **GET, POST, and DELETE**) when using the API Gateway REST API, and that each method is properly mapped to its corresponding database operation. Students are expected to implement a genuinely **RESTful API**, rather than routing all actions through a generic POST request.

Report

In your report, you must include an AWS architecture diagram that clearly illustrates the overall system design you consider most appropriate for this application scenario.

You are also required to describe and justify your frontend and backend architectural decisions, explaining how the components interact within the AWS environment.

Submission Format

Your assignment should follow the requirement below and submit via Canvas>Assignments>Assessment 2.

When you submit work electronically, you agree to the assessment declaration.

Create a **.zip** file containing the source code you developed (individual programs, the code running on the Apache/Container, and your Lambda functions). **You do not need to submit the API gateway configuration.**

Failure to follow the format requirements incurs up to 4 marks penalty.

Name your report and work log using the following format:

- GroupLeaderStudentID_Group<GroupNumber>.zip
- GroupLeaderStudentID_Group<GroupNumber>_**report**.pdf
- GroupLeaderStudentID_Group<GroupNumber>_**worklog**.pdf

Replace:

- GroupLeaderStudentID with **the student number of the group leader** (e.g., s1234567)
- <GroupNumber> with **your group number** (e.g., 3)

Example:

`s1234567_Group3.zip`

`s1234567_Group3_report.pdf`

`s1234567_Group3_worklog.pdf`

Note: The group leader is responsible for submitting the group's final files. Duplicate submissions from the same group will not be accepted.

Your assessment will NOT be allowed to be demonstrated until you submit in Canvas.

- You will need to demonstrate your project online via Teams by scheduling an appointment with a tutor in Week 9.

- All demonstrations must be completed during Week 10 and Week 11. There will be penalties if you fail to complete and demonstrate your work by 24/05 unless you have a valid extension.
- The demo will take 20 minutes for each group.
- Each group is allowed only one demonstration session; overbooking additional slots will result in a 10% penalty.
- The group should collaboratively decide which member(s) will lead the demonstration.

Academic Integrity and Plagiarism (standard warning)

Academic integrity is about honest presentation of your academic work. It means acknowledging the work of others while developing your own insights, knowledge, and ideas. You should take extreme care that you have:

- Acknowledged words, data, diagrams, models, frameworks and/or ideas of others you have quoted (i.e., directly copied), summarized, paraphrased, discussed, or mentioned in your assessment through the appropriate referencing methods,
- Provided a reference list of the publication details so your reader can locate the source if necessary. This includes material taken from Internet

If you do not acknowledge the sources of your material, you may be accused of plagiarism because you have passed off the work and ideas of another person without appropriate referencing, as if they were your own.

RMIT University treats plagiarism as a very serious offence constituting misconduct. For further information on our policies and procedures, please refer to the **University website.**